PROJECT MISSION STATEMENT

Magic: The Gathering — Web-Based Commander Simulator

1. Project Overview and Primary Objective

We are starting this project completely from scratch.

The objective is to design, develop, test, and deploy a fully functional, web-based Magic: The Gathering (MTG) Commander game that accurately reproduces the mechanics, rules, cards, abilities, and interactions of real-world Magic: The Gathering.

The game must allow users to build, edit, save, and play custom Commander decks against human opponents, AI-controlled opponents, or a combination of both.

The finished application must provide a complete, interactive Commander experience, including:

- Full support for 2-, 3-, and 4-player Commander games.
- Online multiplayer with private rooms and multiple devices.
- AI-controlled opponents that automatically fill unoccupied seats.
- A comprehensive MTG card catalog with images and current Oracle text.
- Accurate implementation of all supported card abilities, effects, and interactions.
- A fully functional, visually appealing battlefield.
- An intuitive deck builder and deck management system.
- An integrated AI deck-building assistant capable of recommending cards, identifying synergies, and generating complete decks.
- AI opponents capable of making intelligent, strategically appropriate decisions.
- Complete game-state management, including turn structure, mana, combat, priority, and the stack.
- Automated game simulations, statistical analysis, and testing.
- A maintainable, scalable architecture supporting future MTG card releases and rules updates.

The ultimate goal is to create a complete, reliable, rules-accurate online Commander application—not merely a demonstration, partial simulator, or prototype.

All development decisions must prioritize correctness, completeness, stability, extensibility, and the quality of the user experience.

---

2. Foundational Rules Engine — XMage and Forge

Rather than attempting to manually recreate the entire Magic: The Gathering rules engine, this project will leverage existing open-source implementations.

The following repositories will serve as the primary development references:

XMage:
https://github.com/magefree/mage

Forge:
https://github.com/Card-Forge/forge

2.1 XMage — Primary Rules Engine

XMage will serve as the preferred foundation for the game's authoritative Java-based MTG rules engine.

The application should reuse and adapt XMage's existing implementations wherever technically and legally appropriate, including:

- Card definitions and card-specific abilities.
- Casting spells and activating abilities.
- Triggered, activated, static, and replacement abilities.
- Mana costs, colored mana, generic mana, hybrid mana, Phyrexian mana, snow mana, and variable X costs.
- Alternative, optional, additional, and discounted casting costs.
- Creatures, artifacts, enchantments, planeswalkers, battles, lands, and other applicable card types.
- All zones and movement between zones.
- Combat and damage assignment.
- Targeting restrictions and protection abilities.
- Copy effects and token generation.
- Counters, proliferate, and modifications to power and toughness.
- The stack, priority, and spell resolution.
- Layer-system interactions and continuous effects.
- State-based actions and turn-based actions.
- Commander-specific mechanics and rules.
- Multiplayer rules, elimination, and victory conditions.

XMage must remain the single authoritative engine responsible for determining whether an action is legal and what its consequences are.

The frontend and AI systems must communicate with the authoritative engine rather than independently deciding or manipulating game rules.

Existing XMage components should be reused where practical instead of being rewritten.

2.2 Forge — Secondary Development Resource

Forge will be used as a secondary implementation and research reference.

Its potential uses include:

- Studying AI decision-making approaches.
- Comparing implementations of complicated card abilities.
- Identifying cards or mechanics missing from XMage.
- Researching alternative card scripting approaches.
- Developing strategic evaluations and opponent behavior.
- Designing automated simulations.
- Comparing expected outcomes for difficult rules interactions.
- Informing the project's testing methodology.

Forge should not operate as a second authoritative rules engine for the same match.

Before copying or integrating any Forge source code, examine its license and compatibility with the project's intended distribution model. Forge's GPL licensing obligations must be respected. Studying its publicly available implementation does not automatically authorize unrestricted reuse of its code.

2.3 Engine Integration Requirements

Before developing the complete application, conduct a detailed technical audit of XMage and Forge.

Determine:

1. How each repository organizes card definitions and abilities.
2. How game states are represented and updated.
3. How actions enter the rules engine.
4. How legal actions and valid targets are identified.
5. How the existing AI interacts with the game engine.
6. How multiplayer games are hosted and synchronized.
7. How hidden information is protected.
8. Whether games can run without a graphical interface.
9. Whether a custom browser-based frontend can control games through a Java API.
10. How new cards and mechanics can be added.
11. Which dependencies and components can safely be reused.
12. What licensing, compatibility, performance, or maintenance limitations exist.

Create a small working integration proof of concept before committing to the full architecture.

If XMage's existing interfaces are unsuitable, develop a carefully isolated adapter or evaluate a different approach. Do not begin rewriting the rules engine simply because integration is inconvenient.

---

3. Complete Card Catalog and Card Functionality

The application must support the full applicable Magic: The Gathering card catalog, including existing cards and future releases.

Card information should be obtained from authoritative and reliable sources.

Use:

- XMage as the primary source of implemented card behavior.
- Forge as a secondary implementation reference.
- Scryfall for card metadata, Oracle text, images, printings, mana costs, legality information, and identifiers.
- The official MTG Comprehensive Rules and relevant rulings for determining correct behavior.

3.1 Card Information

Every card must have the relevant attributes available to the application, including:

- Card name and unique identifier.
- Oracle text.
- Mana cost and mana value.
- Color and color identity.
- Card types, supertypes, and subtypes.
- Power, toughness, and loyalty where applicable.
- Defense values for battles where applicable.
- Keywords and abilities.
- Card images and alternate printings.
- Set information and collector number.
- Format legality.
- Commander eligibility.
- Related cards, tokens, and transformed faces.
- Face-specific information for split, modal double-faced, transforming, adventure, and other multi-part cards.

Card metadata must remain separate from the rules execution layer.

3.2 Required Card Mechanics

The rules engine must correctly support all applicable MTG mechanics, including but not limited to:

- Flashback, escape, jump-start, retrace, aftermath, and other graveyard casting mechanics.
- Cycling, channel, discard abilities, and activated abilities.
- Suspend, foretell, plot, adventure, rebound, and other alternative casting or timing mechanics.
- Prepared spells and other applicable set-specific mechanics, subject to their actual official rules.

- Discover, cascade, storm, replicate, conspire, and spell copying.
- Convoke, delve, improvise, affinity, offering, emerge, and other cost-changing mechanics.
- Kicker, multikicker, buyback, overload, entwine, and additional costs.
- X spells and X-dependent effects.
- Modal spells and multiple target selections.
- Tutoring, searching, revealing, looking at, and shuffling libraries.
- Creature tokens, noncreature tokens, token copies, and token doublers.
- Equipment, auras, vehicles, and attachment restrictions.
- Transforming, morph, manifest, disguise, cloak, and face-down interactions.
- Replacement effects, prevention effects, and triggered abilities.
- +1/+1 counters, -1/-1 counters, loyalty counters, poison counters, and other counter types.
- Proliferate and counter multiplication.
- Exile, blinking, phasing, and zone-change effects.
- Landfall and other event-triggered abilities.
- Tribal and creature-type interactions.
- Alternate win conditions and loss conditions.
- Continuous effects and the official layer system.
- Commander-specific interactions.

3.3 Card-Specific Effects

The engine must also support unusual or highly specific cards.

For example, a card that:

- Reduces costs based on the number of creatures of a particular type.
- Modifies spells cast from a particular zone.
- Changes how counters are placed.
- Creates replacement effects involving tokens.
- Changes the number or characteristics of spells cast.
- Grants abilities to particular creatures.
- Modifies the game's rules.
- Changes a player's win or loss conditions.
- Interacts with previously cast spells, turn history, or other game-state information.

These interactions must be executed correctly using the authoritative rules engine.

Do not treat displaying accurate Oracle text as proof that a card is implemented.

A card is considered functionally supported only when its relevant abilities and game interactions are executable, legal, and verified.

Cards lacking implementation must be identified through a coverage system and must never be silently represented as fully functional.

---

4. Commander Game Rules and Gameplay

The game will initially focus on the Commander format.

It must correctly implement Commander-specific rules, including:

- 100-card deck construction.
- Singleton restrictions and their exceptions.
- Commander color identity.
- Current Commander banned cards.
- Starting life totals.
- Commander tax.
- Commander damage.
- Command zone interactions.
- Partner and other valid multi-commander configurations.
- Backgrounds and other special commander qualifications.
- Commander replacement and zone-change rules.
- Multiplayer turn order and priority.
- Multiplayer combat.
- Commander-specific victory and elimination rules.

Official game rules and legality information should be versioned and updated as necessary.

4.1 Player Configurations

Support games with:

- 2 total players.
- 3 total players.
- 4 total players.

Allow any supported combination of human and AI players, with at least one human player for normal interactive games.

Examples:

- 1 human + 1 AI.
- 1 human + 2 AI.
- 1 human + 3 AI.
- 2 humans + 2 AI.
- 3 humans + 1 AI.
- 4 humans.

Human players should be able to play from separate devices.

4.2 Standard Game Operations

Implement:

- Deck shuffling.
- Opening hands and mulligans.
- Choosing first player.
- Untap, upkeep, draw, main, combat, second main, and ending phases.
- Priority passing and holding priority.
- Playing lands and casting spells.
- Selecting modes and targets.
- Paying costs and choosing mana.
- Resolving spells and abilities.
- Declaring attackers and blockers.
- Combat damage assignment.
- Triggered ability ordering.
- Responding to spells and abilities.
- Correct handling of multiple simultaneous effects.
- Player elimination.
- Game completion and winner identification.

Automatic passing and shortcut systems must never prevent a player from responding when the rules permit a response.

---

5. Online Multiplayer System

The application must support browser-based multiplayer.

5.1 Room Creation and Joining

A user must be able to:

1. Create a new Commander room.
2. Choose the room's player limit.
3. Configure room privacy and game settings.
4. Generate a room code or invitation link.
5. Share that room with other players.
6. Allow players to join from different devices.
7. Allow each player to select a saved or imported Commander deck.
8. Fill remaining seats with AI-controlled players.
9. Start the game when the host is ready and required conditions are met.

5.2 Multiplayer Synchronization

All player actions must be synchronized through a server-authoritative game state.

The system must:

- Validate actions server-side.
- Prevent illegal moves.
- Enforce priority.
- Preserve the stack.
- Protect hidden information.
- Synchronize life totals, permanents, tokens, counters, mana pools, and all other game-state information.
- Prevent duplicated or conflicting actions.
- Handle disconnections.
- Allow players to reconnect to an existing game.
- Handle player concessions.
- Maintain complete game logs.
- Prevent spectators from accessing hidden information.

The multiplayer system must remain stable even when several players act or submit commands close together.

5.3 Multiplayer User Experience

The frontend must clearly indicate:

- Whose turn it is.
- Who currently has priority.
- Which player controls each permanent.
- Which spells or abilities are on the stack.
- Which targets have been selected.
- Which players are attacking whom.
- Which players have passed priority.
- Which players are human or AI.

---

6. AI Opponent System

AI opponents must play according to the same rules as human players.

They should not have access to hidden information that their player would not legally know.

They must make decisions through legal-action interfaces exposed by the game engine.

6.1 AI Decision-Making

AI players should evaluate:

- Current board state.
- Available mana.
- Cards in hand.
- Legal plays.
- Strategic objectives.
- Commander abilities and synergies.
- Opponent threats.
- Removal priorities.
- Combat probabilities.
- Mana efficiency.
- Card advantage.
- Potential combinations and win conditions.
- Defensive requirements.
- Whether to hold up interaction.
- Whether to advance or delay their game plan.

6.2 AI Personalities

Support configurable AI behaviors, such as:

- Aggressive.
- Defensive.
- Control-oriented.
- Combo-oriented.
- Value-oriented.
- Balanced.
- Political or negotiation-aware.

AI behavior must be grounded in the game state and the AI's available information.

The game should initially leverage existing XMage AI functionality where appropriate and gradually extend it with custom strategic decision-making.

AI difficulty should improve through evaluation, testing, and structured decision algorithms rather than relying exclusively on free-form language model predictions.

6.3 AI Performance and Reliability

AI players must:

- Choose legal actions.
- Resolve all mandatory decisions.
- Avoid infinite decision loops.
- Handle complicated board states.
- Avoid unnecessary delays.
- Recognize available win conditions.
- Respond to threats appropriately.

- Play complete games without manual intervention.

Where applicable, decision-making should include configurable time budgets and deterministic fallback behavior.

---

7. Deck Builder and Deck Management

The game must contain a complete Commander deck-building environment.

7.1 Deck Creation

Users must be able to:

- Choose one or more legal commanders.
- Search the full card catalog.
- Filter by color identity, card type, mana value, abilities, sets, and other properties.
- Add and remove cards.
- Change printings and card art.
- View mana curves.
- View color distribution.
- View card-type distribution.
- Track land counts.
- Track card categories.
- Validate deck legality.
- Import decklists.
- Export decklists.
- Save decks.
- Duplicate decks.
- Rename decks.
- Edit previously saved decks.
- Delete decks.

The application should offer a visually intuitive deck editor with drag-and-drop or similarly convenient interactions.

7.2 Deck Evaluation

Provide deck analysis including:

- Mana curve.
- Ramp density.
- Card draw.
- Interaction.
- Board wipes.

- Protection.
- Mana-base reliability.
- Creature counts.
- Synergy density.
- Win conditions.
- Combo packages.
- Commander compatibility.
- Likely weaknesses.
- Estimated power level, with clear limitations.
- Commander Bracket evaluation based on current official guidance where available.

Deck evaluation must distinguish objective measurements from heuristic estimates.

---

8. AI Deck-Building Assistant

The deck builder must include an AI assistant.

The assistant should be capable of:

- Recommending commanders.
- Identifying a commander's primary strategies.
- Finding cards with strong mechanical synergy.
- Suggesting efficient mana bases.
- Identifying important missing card categories.
- Suggesting additions and cuts.
- Recommending win conditions.
- Evaluating combos.
- Explaining card interactions.
- Identifying redundancy.
- Recommending budget alternatives.
- Suggesting upgrades for a target power level.
- Building an entire Commander deck from a user prompt.

8.1 Automatic Deck Generation

Users must be able to request a complete deck based on inputs such as:

- Commander.
- Preferred strategy.
- Theme.
- Budget.
- Power level or bracket.
- Preferred win conditions.

- Required cards.
- Excluded cards.
- Desired land count.
- Casual or competitive intent.

The AI must generate a complete legal decklist and submit it to a deterministic deck validator.

The assistant must not invent cards, recommend cards outside the commander's color identity, or ignore deck-construction restrictions.

8.2 Synergy Analysis

AI suggestions should use a combination of:

- Card metadata.
- Card abilities.
- Shared mechanics.
- Commander interactions.
- Card-type relationships.
- Creature-type relationships.
- Combo information.
- Mana curve and deck composition.
- Strategic heuristics.
- Optional gameplay simulation results.

All recommendations should include explanations.

---

9. User Interface and Experience

The application must have a polished, responsive, modern user interface.

The interface must work well on common desktop and laptop browsers, with responsive support for smaller screens where practical.

9.1 Battlefield

Display:

- All participating players.
- Life totals.
- Commanders.
- Battlefield permanents.
- Lands.
- Creatures.

- Artifacts.
- Enchantments.
- Planeswalkers.
- Battles.
- Tokens.
- Counters.
- Equipment and attachments.
- Graveyards.
- Exile zones.
- Libraries.
- Player hands where the user has permission to view them.
- The stack.
- Turn and phase indicators.
- Priority indicators.
- Available mana.
- Combat assignments.

Cards should remain readable, selectable, and accessible without hiding important game zones.

9.2 Interactive Gameplay

Users must be able to:

- Click or select cards.
- Inspect enlarged card images.
- Read Oracle text.
- View relevant game-state modifications.
- Cast spells.
- Activate abilities.
- Select targets.
- Choose modes.
- Pay mana and other costs.
- Assign attackers.
- Assign blockers.
- Respond to spells and abilities.
- Pass priority.
- Use supported gameplay shortcuts.
- View game history.

9.3 Presentation Standards

The interface should prioritize:

- Readability.
- Intuitive interaction.
- Clear visual hierarchy.

- Smooth but optional animations.
- Responsive layouts.
- Accessibility.
- Consistent visual design.
- Fast interactions.
- Minimal unnecessary clicks.

The appearance of the game must never compromise the visibility or usability of the battlefield.

---

10. Game Simulation and Statistical Analysis

The application must contain an automated simulation subsystem.

This subsystem should run complete MTG games using the same authoritative rules engine as normal gameplay.

It must support:

- AI-versus-AI matches.
- Two-, three-, and four-player simulations.
- Batch simulations.
- Round-robin competitions.
- Elimination tournaments.
- Randomized player seating.
- Randomized starting players.
- Reproducible random seeds.
- Configurable AI strategies.

10.1 Required Statistics

Track:

- Games played.
- Wins.
- Losses.
- Win rate.
- Average game length.
- Median game length.
- Turn of victory.
- Winning player and deck.
- Identified win conditions.
- Most influential cards.
- Commander cast frequency.
- Mana development.

- Mulligan frequency.
- Card draw.
- Removal usage.
- Interaction frequency.
- Player elimination order.
- Matchups and matchup-specific performance.

Win-condition attribution should be evidence-based, using actual game events rather than unsupported assumptions.

The subsystem should support large simulations, including tens of thousands of games, subject to available computing resources.

---

11. Architecture and Technology Requirements

The project should use a modular architecture with clearly separated responsibilities.

11.1 Backend

Preferred technologies:

- Java.
- XMage integration.
- Spring Boot for the application API.
- WebSockets for real-time communication.
- PostgreSQL for persistent application data.
- Java-based AI services.
- Containerized deployment.

The backend should provide independent services or modules for:

- Game creation.
- Game state.
- Player management.
- Room management.
- Deck management.
- Card catalog synchronization.
- AI decisions.
- Simulation.
- Statistics.
- Authentication and authorization.
- Persistence.

Avoid duplicating rules logic outside the authoritative engine.

11.2 Frontend

Preferred technologies:

- React.
- TypeScript.
- Vite.
- Responsive CSS and reusable UI components.

The frontend must not directly modify authoritative game state.

11.3 Card Data Management

Card metadata must be stored separately from game-state data.

The system should support:

- Card catalog imports.
- Oracle updates.
- Image and printing references.
- Card identifier mappings.
- Rules implementation mappings.
- Missing-card detection.
- Version tracking.
- Database migrations.

11.4 Security and Integrity

Include:

- Authenticated user sessions.
- Server-side action validation.
- Strict authorization checks.
- Hidden-information protection.
- Secure room invitations.
- Rate limiting.
- Input validation.
- Safe database operations.
- Protection against unauthorized game-state manipulation.
- Reliable logging and error handling.

---

12. Missing Mechanics and Card Coverage

A central requirement is the ability to detect and correct incomplete card implementations.

Create an automated card coverage system that distinguishes:

1. Cards with verified complete implementation.
2. Cards with implementation but incomplete verification.
3. Cards with known unsupported interactions.
4. Cards missing implementation.
5. Cards that cannot legally appear in the selected format.

The system should cross-reference the card catalog against available engine implementations.

It should also identify:

- Unsupported keywords.
- Missing triggered abilities.
- Unsupported replacement effects.
- Incorrect targeting behavior.
- Incomplete cost handling.
- Missing token definitions.
- Incorrect Commander interactions.

When XMage does not support a particular card or interaction, develop a targeted extension, preferably in an isolated compatibility module.

Do not silently replace unimplemented card abilities with simplified approximations.

The target is complete functional coverage of the applicable catalog, verified through increasingly comprehensive automated tests.

---

13. Testing and Quality Assurance

Testing is a mandatory part of development.

Every feature must have clearly defined acceptance criteria.

The testing system must include:

- Unit tests.
- Integration tests.
- Rules interaction tests.
- Card-specific regression tests.
- Multiplayer synchronization tests.
- Hidden-information security tests.
- AI decision tests.
- Deck legality tests.

- Browser interface tests.
- End-to-end game tests.
- Performance tests.
- Automated full-game simulations.

Tests must verify actual gameplay outcomes rather than merely confirming that code executes without errors.

For example, a card with a triggered ability must be tested to ensure that:

1. The correct event triggers the ability.
2. The ability enters the stack when required.
3. Relevant players receive priority.
4. Targets and choices are handled correctly.
5. The effect resolves properly.
6. The resulting game state is valid.

Maintain a regression suite so that implementing one card or mechanic does not break previously functioning interactions.

---

14. Development Process and AI Implementation Instructions

IMPORTANT: THIS PROJECT WILL BE DEVELOPED WITH THE ASSISTANCE OF AI CODING AGENTS.

The AI must follow a disciplined, incremental development process.

Do not attempt to generate the entire application in a single unverified implementation.

Development must proceed through clearly defined phases and individual steps.

Each step must build directly on previously completed work.

14.1 Mandatory Development Rules

For every implementation step:

1. Examine the existing project structure and relevant source code.
2. Identify the specific requirements being implemented.
3. Investigate existing XMage functionality before writing replacement code.
4. Implement the functionality using maintainable, modular architecture.
5. Integrate the implementation into the actual application.
6. Write automated tests covering successful, invalid, and edge-case behavior.

7. Execute the relevant tests.
8. Fix detected errors.
9. Repeat testing until the acceptance criteria pass.
10. Update technical documentation.
11. Record remaining limitations and unsupported behavior.
12. Preserve compatibility with previously completed functionality.

A step is not complete merely because code has been written.

It is complete only when the functionality is integrated, demonstrably operational, and passes its required tests.

14.2 Continuous Work Within Each Step

The AI coding agent must continue working through the implementation, debugging, integration, and verification tasks within the current execution session until the step satisfies its completion criteria or a genuine external blocker prevents further progress.

Do not prematurely declare success.

Do not present partially implemented or untested features as complete.

Do not skip difficult mechanics or substitute placeholder behavior without explicitly identifying the limitation.

If an implementation fails a test, investigate the failure and correct the underlying issue.

If a dependency or architectural constraint blocks progress, document the issue and present a technically justified solution.

Never fabricate test results, card coverage percentages, completed functionality, or successful integrations.

14.3 Progress Tracking

Maintain:

- An implementation roadmap.
- A current-step checklist.
- A completed-feature register.
- A known-issues register.
- A card coverage report.
- A regression test report.
- Architecture documentation.
- A dependency and licensing register.
- A record of major technical decisions.

Each development step should produce a working project state that can be tested independently.

14.4 Completion Reporting

At the end of every step, provide:

- A summary of implemented functionality.
- The specific components and files changed.
- Tests executed.
- Test results.
- Current functionality.
- Remaining limitations.
- Any architectural decisions.
- The next required development step.

Continue implementing and fixing the current step where possible rather than asking for unnecessary confirmation or declaring completion prematurely.

---

15. Required Development Order

Development should follow this dependency-aware sequence:

Phase 1 — Repository and Architecture Audit

Evaluate XMage and Forge, verify licenses, establish the technical architecture, and identify reusable components.

Phase 2 — Backend Foundation

Establish the Java project, database architecture, dependency management, testing framework, and development environment.

Phase 3 — XMage Integration

Create a working headless rules-engine integration with programmatic game creation, legal-action submission, and game-state retrieval.

Phase 4 — Commander Game Functionality

Verify 2-, 3-, and 4-player games, Commander-specific rules, player actions, AI seats, and complete match execution.

Phase 5 — Card Catalog Integration

Integrate Scryfall metadata, images, identifiers, legality data, and engine support tracking.

Phase 6 — Game API and State Management

Develop secure APIs for actions, game states, priority, valid targets, card inspection, and player-specific information.

Phase 7 — Browser-Based Battlefield

Build the responsive, interactive React interface and connect it to real game logic.

Phase 8 — Multiplayer Rooms

Implement room creation, invitations, joining, synchronization, human/AI seating, reconnection, and game completion.

Phase 9 — Deck Builder and Persistence

Implement deck creation, editing, saving, importing, exporting, validation, and statistical deck analysis.

Phase 10 — AI Opponent Development

Improve AI strategies, board-state evaluation, decision-making, combat, interaction, and multiplayer behavior.

Phase 11 — AI Deck-Building Assistant

Develop synergy recommendations, deck evaluations, card substitutions, and complete deck generation.

Phase 12 — Rules and Card Coverage Expansion

Identify unsupported cards and mechanics, implement missing behavior, and expand automated rules verification.

Phase 13 — Simulation and Analytics

Implement headless simulations, tournament formats, comprehensive statistics, and performance optimization.

Phase 14 — UI and User Experience Refinement

Complete responsive layouts, card interactions, animations, accessibility, settings, and performance improvements.

Phase 15 — Full Integration and Quality Assurance

Perform comprehensive end-to-end tests, multiplayer stress tests, regression testing, security validation, and compatibility verification.

Phase 16 — Deployment and Release

Package and deploy the web application and Java backend, configure the database, document operations, and verify a production-like environment.

A phase is complete only when all of its documented acceptance criteria are satisfied.

---

16. Definition of a Fully Completed Project

The project will be considered complete only when all of the following are true:

- Users can access the game through a web browser.
- Users can create accounts and save their decks.
- Users can build, view, edit, import, and export Commander decks.
- The card catalog contains the full intended MTG catalog.
- Applicable Commander-legal cards have verified functional implementations, including special mechanics and interactions.
- Users can play complete 2-, 3-, and 4-player Commander games.
- Players can create online rooms and join from different devices.
- Empty seats can be filled by AI opponents.
- AI opponents can play complete games legally and make strategically meaningful decisions.
- The battlefield correctly reflects the authoritative game state.
- The rules engine correctly handles priority, stack interactions, combat, mana, abilities, and Commander-specific rules.
- Hidden information is appropriately protected.
- The AI deck assistant can recommend cards and generate valid Commander decks.
- Automated simulations produce reproducible game results and meaningful statistics.
- Automated regression tests pass.
- Multiplayer reconnection and failure handling work correctly.
- The application performs acceptably under the project's defined load requirements.
- Card updates and future sets can be integrated without requiring a complete architectural redesign.
- The application can be deployed and operated reliably.
- Remaining limitations are documented and no feature is falsely represented as complete.

The final product must be a functional game, not merely a repository containing implementations of individual features.

---

17. First Instructions to the AI Development Agent

Before writing game code:

1. Review the complete mission statement.
2. Examine the XMage and Forge repositories.
3. Review their architecture, card implementations, AI systems, dependencies, and licensing.
4. Determine whether XMage can be adapted to power a headless, browser-controlled Commander game.
5. Identify any technical or legal blockers.
6. Propose the final system architecture.
7. Produce a complete project directory structure.
8. Create a detailed development roadmap.
9. Divide each phase into numbered, independently testable implementation steps.
10. Establish objective acceptance criteria for every step.

The initial implementation should demonstrate a functioning XMage-backed Commander game that can be created and controlled programmatically through a Java interface.

Do not prioritize visual design until the fundamental engine integration has been demonstrated.

Do not begin independently recreating existing card mechanics when a compatible, licensed, maintainable implementation already exists.

Development philosophy: Reuse proven rules implementations, build custom functionality where it adds value, verify everything, and never confuse partial functionality with project completion.

18. Additional Mandatory Architectural Requirements

The following three decisions are **binding project requirements**, not optional recommendations. They supplement Sections 2, 6, 8, 10-16, and 17 of this mission statement. If an implementation plan conflicts with them, these requirements take precedence.

18.1 Interface-Independent Game Engine and Headless Execution

The complete game must operate independently of the graphical user interface. The authoritative XMage-based Java rules engine and application services must run without React, a browser, a human player, or any rendering process.

- Expose the same validated game-creation, legal-action, choice/target submission, game-state, and completion interfaces to browser clients, AI controllers, and automated simulation workers.
- Use a player/controller abstraction so human inputs and AI actions pass through the same authoritative legality and resolution pipeline. Neither may mutate game state directly.
- Support 2-, 3-, and 4-player games in interactive and fully headless modes, including AI-only games with deterministic seeds and event logs.
- Ensure headless operation does not require GUI dependencies, displays, simulated mouse clicks, or frontend automation.

Acceptance tests: A browser-based Commander game and a headless Commander game must execute through the same rules engine and game-action interfaces. A complete 4-AI match must finish without any UI process. With the same decks, seed, and action choices, engine-level results must be reproducible.

18.2 Two Separate AI Systems with Independent Responsibilities

The project must implement two **distinct, modular AI subsystems**. They may share card metadata, synergy features, and evaluation utilities but must not be treated as one monolithic AI service.

- **Gameplay AI:** controls an in-game seat, observes only legally available information, evaluates legal actions, handles choices and priority, and plays Commander strategically. It must use the rules engine's validated action interface and have timeouts and safe fallback behavior.
- **Deck-Building AI:** recommends cards, explains synergies and weaknesses, proposes changes, and generates full Commander decks outside a live game. Every output must be checked by a deterministic card-catalog lookup and deck legality validator before it can be saved or used.
- Keep their APIs, tests, performance budgets, model/algorithm choices, and deployment lifecycles separately configurable. A failure of deck recommendations must not interrupt an active game.

Acceptance tests: Gameplay AI must complete a headless game with legal moves and no access to opponents' hidden information. Deck-Building AI must independently produce an importable, legal 100-card Commander deck; intentionally invalid recommendations must be rejected with clear explanations.

18.3 Evidence-Based, Auditable Full Card Coverage

Full catalog presence, successful card loading, displayed Oracle text, or inherited XMage/Forge coverage claims **do not establish complete functional support**. Full support is a measurable end-state that must be demonstrated and continually revalidated.

- Maintain a versioned per-card coverage registry mapped to canonical card identifiers, engine implementation, supported mechanics, test cases, rules version, and verification status.
- Distinguish metadata-only, implemented-but-unverified, partially supported, known broken, and verified-supported cards; separately record cards outside Commander legality.
- Run automated mechanic-level, card-specific, regression, and representative cross-card interaction tests. Record actual results, known gaps, and changes following rules updates or new card releases.
- Block unsupported cards or explicitly restrict their use in rules-accurate games; never silently approximate abilities or report incomplete interactions as fully implemented.

Acceptance tests: A generated coverage report must list the entire applicable catalog, evidence for every verified-supported card, any unsupported interaction, and the exact tested engine/catalog/rules versions. The full-coverage completion gate in Section 16 can pass only after required tests demonstrate functional support for every in-scope Commander-legal card.

18.4 Development Plan and Definition-of-Done Updates

- Treat a public web game and an automated headless game as two clients of one engine, not two rules implementations. Validate this separation during Phases 3, 4, 6, 7, and 13.
- Develop gameplay AI and deck-building AI as separate milestones with separate completion reports in Phases 10 and 11.
- Generate and review the card coverage registry during Phase 5, then continuously test it through Phases 12 and 15 and before releases.
- No phase or final release may claim completion without running its objective integration and acceptance tests and documenting unresolved gaps.

Final completion additions to Section 16: The application runs complete Commander games without a browser; both AI systems operate independently with their respective validation; and a versioned, evidence-backed coverage report demonstrates functionality of every in-scope Commander-legal card.